

Troubleshooting Guide

Common Issues and Solutions

Signaling Server Issues

Issue: Server won't start

Symptoms:

- Port already in use error
- Permission denied error
- Module not found errors

Solutions:

1. Port already in use:

```
# Find process using port 3000
sudo lsof -i :3000
# Kill the process
sudo kill -9 <PID>
# Or use different port
PORT=3001 npm start
```

1. Permission denied (port < 1024):

```
# Use port > 1024 or run with sudo (not recommended)
PORT=3000 npm start
```

1. Module not found:

```
# Reinstall dependencies
rm -rf node_modules package-lock.json
npm install
npm run build
```

Issue: WebSocket connections failing

Symptoms:

- Client can't connect
- "Connection refused" errors
- CORS errors

Solutions:

1. Check server is running:

```
curl http://localhost:3000/health
```

1. Verify CORS configuration:

```
# .env file
ALLOWED_ORIGINS=http://localhost:5173,https://yourdomain.com
```

1. Check firewall:

```
sudo ufw status
sudo ufw allow 3000/tcp
```

1. Verify WebSocket upgrade:

```
# Test WebSocket connection
wscat -c ws://localhost:3000/socket.io/?transport=websocket
```

Issue: Sessions not being cleaned up

Symptoms:

- Memory usage increasing over time
- Expired sessions still listed

Solutions:

1. Check cleanup interval:

```
CLEANUP_INTERVAL_MS=60000 # 1 minute
SESSION_TIMEOUT_MS=3600000 # 1 hour
```

1. Manually trigger cleanup:

```
# Restart server
sudo systemctl restart signaling-server
```

1. Monitor memory:

```
# Check Node.js memory usage
node --max-old-space-size=2048 dist/index.js
```

VM Host Issues

Issue: Service won't start

Symptoms:

- systemd service fails

- Python errors
- Import errors

Solutions:

1. Check service status:

```
sudo systemctl status vm-host
sudo journalctl -u vm-host -n 50
```

1. Test manually:

```
cd /opt/cloud-gaming-vm-host
source venv/bin/activate
python3 -m src.main
```

1. Check dependencies:

```
pip install -r requirements.txt --upgrade
```

1. Verify Python version:

```
python3 --version # Should be 3.8+
```

Issue: Can't capture screen

Symptoms:

- Black screen on client
- FFmpeg errors
- "Cannot open display" errors

Solutions:

1. Verify X11 display:

```
echo $DISPLAY # Should show :0 or similar
xdpyinfo # Should show display info
```

1. Check X11 permissions:

```
xhost +local:
# Or add user to video group
sudo usermod -a -G video $USER
```

1. Test FFmpeg capture:

```
ffmpeg -f x11grab -video_size 1920x1080 -framerate 30 -i :0 -t 5 test.mp4
# Should create test.mp4
```

1. For headless servers (no physical display):

```
# Install virtual display
sudo apt-get install xvfb
# Start virtual display
Xvfb :99 -screen 0 1920x1080x24 &
export DISPLAY=:99
# Update .env
DISPLAY=:99
```

Issue: High CPU usage

Symptoms:

- CPU at 100%
- Lag and frame drops
- System unresponsive

Solutions:

1. Enable hardware encoding:

NVIDIA:

```
# Verify NVENC support
nvidia-smi
# Update screen_capture.py to use h264_nvenc
```

AMD:

```
# Install drivers
sudo apt-get install mesa-va-drivers
# Use h264_amf codec
```

1. Lower quality preset:

```
# config/config.yaml
bitrate:
  initial: "medium" # Instead of "high"
```

1. Reduce FPS:

```
video:
  fps: 30 # Instead of 60
```

1. Check other processes:

```
htop
# Kill unnecessary processes
```

Issue: WebRTC connection failed

Symptoms:

- "Connection failed" in logs
- No video stream
- ICE candidates not gathering

Solutions:

1. Check signaling server connection:

```
# Test connectivity
curl https://signaling.yourdomain.com/health
# Check logs
sudo journalctl -u vm-host | grep "Connected to signaling"
```

1. Verify STUN servers:

```
# Test STUN server
sudo apt-get install stun-client
stunclient stun.l.google.com 19302
```

1. Check firewall (WebRTC uses UDP):

```
# Allow UDP ports for WebRTC
sudo ufw allow 10000:20000/udp
```

1. Check NAT traversal:

```
# Add TURN server if behind strict NAT
network:
  turn_servers:
    - urls: "turn:turn.yourdomain.com:3478"
      username: "user"
      credential: "password"
```

Issue: Input not working

Symptoms:

- Keyboard/mouse not responding
- Gamepad not detected on VM
- Input lag

Solutions:

1. Check input handler:

```
# Verify pynput is installed
pip list | grep pynput
```

1. Check X11 permissions:

```
# Allow input injection
xhost +local:
```

1. Test input manually:

```
from pynput.keyboard import Key, Controller
keyboard = Controller()
keyboard.press('a')
keyboard.release('a')
```

1. For Wayland (not supported):

```
# Switch to X11
sudo nano /etc/gdm3/custom.conf
# Uncomment: WaylandEnable=false
sudo systemctl restart gdm3
```

Web Client Issues

Issue: Video not playing

Symptoms:

- Black screen
- “Waiting for stream” message
- Video element shows no content

Solutions:

1. Check browser console:

```
// Open DevTools (F12) and check for errors
console.log errors related to WebRTC
```

1. Check autoplay policy:

```
// Browsers may block autoplay
videoRef.current?.play().catch(err => {
  console.error('Autoplay blocked:', err);
  // Show play button for user interaction
});
```

1. Verify video element:

```
// Check video element has stream
console.log(videoRef.current.srcObject);
```

1. Check WebRTC connection:

```
// Verify peer connection state
console.log(peerConnection.connectionState);
console.log(peerConnection.iceConnectionState);
```

Issue: High latency

Symptoms:

- Input lag
- Delayed video
- Poor responsiveness

Solutions:

1. Check connection stats:

```
// Monitor stats in real-time
stats.latency // Should be < 100ms
stats.packetLoss // Should be < 5%
```

1. Network troubleshooting:

```
# Test ping to server
ping signaling.yourdomain.com
# Check bandwidth
speedtest-cli
```

1. Lower quality:

```
// Switch to lower quality preset
changeQuality('medium'); // or 'low'
```

1. Use wired connection:

- WiFi adds 10-50ms latency
- Ethernet is much more stable

2. Check browser performance:

- Close other tabs
- Disable browser extensions
- Update browser to latest version

Issue: Gamepad not detected

Symptoms:

- “No gamepad detected” message
- Controller not responding
- Buttons not registering

Solutions:

1. Check browser support:

```
if ('getGamepads' in navigator) {
  console.log('Gamepad API supported');
} else {
  console.log('Gamepad API not supported');
}
```

1. Test gamepad:

```
// Press any button on controller
window.addEventListener('gamepadconnected', (e) => {
  console.log('Gamepad connected:', e.gamepad);
});
```

1. Try different browser:

- Chrome/Edge: Best support
- Firefox: Good support
- Safari: Limited support

2. Reconnect controller:

- Unplug and replug USB controller
- Or reconnect Bluetooth controller

Issue: Mouse not locked

Symptoms:

- Mouse cursor escapes video area
- Can't control camera in game
- Pointer lock not working

Solutions:

1. Click to lock:

```
// Click on video element to request pointer lock
videoElement.requestPointerLock();
```

1. Check browser permissions:

- Some browsers require user gesture
- Try full-screen mode first

2. Press ESC to release:

- ESC key releases pointer lock
- Re-click video to lock again

3. Browser compatibility:

```
// Check pointer lock support
if ('pointerLockElement' in document) {
  console.log('Pointer Lock supported');
}
```

Network Issues

Issue: Connection drops frequently

Symptoms:

- Stream keeps disconnecting
- “Connection lost” messages
- Reconnection loops

Solutions:

1. Check network stability:

```
# Test packet loss
ping -c 100 signaling.yourdomain.com
# Check for packet loss % at end
```

1. Increase heartbeat frequency:

```
# config/config.yaml (VM host)
heartbeat:
  interval: 10 # More frequent heartbeats
```

1. Check firewall timeouts:

```
# Ensure UDP traffic not blocked
sudo iptables -L -n -v
```

1. Use TURN server:

```
# For restrictive firewalls
network:
  turn_servers:
    - urls: "turn:turn.yourdomain.com:3478"
      username: "user"
      credential: "password"
```

Issue: Poor video quality despite good connection

Symptoms:

- Blocky/pixelated video
- Compression artifacts
- Low framerate

Solutions:

1. Increase bitrate:

```
# config/config.yaml
bitrate:
  high: 12000 # Increase from 8000
```

1. Check encoding settings:

```
# screen_capture.py
preset = "veryfast" # Try different preset
tune = "zerolatency" # Keep for low latency
```

1. Enable hardware encoding:

- NVENC (NVIDIA)
- AMF (AMD)
- Quick Sync (Intel)

2. Check VM resources:

```
# Monitor CPU/GPU usage
htop
nvidia-smi # For NVIDIA GPU
```

SSL/TLS Issues

Issue: SSL certificate errors

Symptoms:

- "Certificate invalid" warnings
- "ERR_CERT_AUTHORITY_INVALID"
- Cannot connect over HTTPS

Solutions:

1. Check certificate validity:

```
openssl s_client -connect signaling.yourdomain.com:443
# Check dates and chain
```

1. Renew Let's Encrypt certificate:

```
sudo certbot renew
sudo systemctl restart nginx
```

1. Verify certificate installation:

```
# Check Nginx config
sudo nginx -t
# Check certificate files
ls -la /etc/letsencrypt/live/yourdomain.com/
```

1. For self-signed certificates (dev only):

```
// Chrome: Allow insecure localhost
// Visit: chrome://flags/#allow-insecure-localhost
```

Debugging Tools

Server-side Debugging

```
# Signaling Server
# Enable debug logging
LOG_LEVEL=debug npm start

# VM Host
# Enable debug logging
LOG_LEVEL=DEBUG python3 -m src.main

# Monitor in real-time
tail -f /opt/cloud-gaming-vm-host/vm-host.log
sudo journalctl -u vm-host -f
```

Client-side Debugging

```
// Enable WebRTC debugging in Chrome
chrome://webrtc-internals/

// Log all WebRTC events
pc.addEventListener('connectionstatechange', () => {
  console.log('Connection state:', pc.connectionState);
});

pc.addEventListener('iceconnectionstatechange', () => {
  console.log('ICE state:', pc.iceConnectionState);
});

pc.addEventListener('icegatheringstatechange', () => {
  console.log('ICE gathering state:', pc.iceGatheringState);
});
```

Network Debugging

```
# Capture network traffic
sudo tcpdump -i any port 3000 -w signaling.pcap

# Monitor WebRTC UDP traffic
sudo tcpdump -i any udp and portrange 10000-20000

# Analyze with Wireshark
wireshark signaling.pcap
```

Performance Profiling

CPU Profiling

```
# Profile Node.js (signaling server)
node --prof dist/index.js
node --prof-process isolate-*.log > profile.txt

# Profile Python (VM host)
python -m cProfile -o vm-host.prof src/main.py
python -m pstats vm-host.prof
```

Memory Profiling

```
# Node.js heap snapshot
node --inspect dist/index.js
# Open chrome://inspect

# Python memory profiling
pip install memory_profiler
python -m memory_profiler src/main.py
```

Log Analysis

Common Error Patterns

Pattern 1: Session not found

```
Error: Session not found or expired
Solution: VM may have restarted, get new session ID
```

Pattern 2: ICE candidates not gathering

```
Warning: No ICE candidates after 5 seconds
Solution: Check STUN/TURN servers, firewall
```

Pattern 3: Frame drops

```
Warning: Dropping frames due to encoding lag
Solution: Enable hardware encoding, lower quality
```

Pattern 4: WebSocket timeout

Error: WebSocket connection timeout
Solution: Check network, firewall, server **load**

Emergency Recovery

Quick Reset

```
# Restart all services
sudo systemctl restart signaling-server
sudo systemctl restart vm-host

# Clear browser cache
# Chrome: Ctrl+Shift+Delete
# Firefox: Ctrl+Shift+Delete

# Flush DNS
sudo systemd-resolve --flush-caches
```

Factory Reset

```
# Signaling Server
cd signaling-server
rm -rf node_modules logs dist
npm install
npm run build

# VM Host
cd /opt/cloud-gaming-vm-host
source venv/bin/activate
pip install -r requirements.txt --force-reinstall

# Restart services
sudo systemctl restart signaling-server vm-host
```

Getting Help

If issues persist:

1. **Check logs thoroughly**
2. **Search existing issues** in repository
3. **Create detailed bug report** with:
 - Error messages
 - Log excerpts
 - System information
 - Steps to reproduce
 - Expected vs actual behavior

Reporting Bugs

Include in bug reports:

```
**Environment:**  
- OS: Ubuntu 22.04  
- Browser: Chrome 120  
- Node.js: 18.17.0  
- Python: 3.10.12  
  
**Issue:**  
[Clear description]  
  
**Steps to Reproduce:**  
1. Step one  
2. Step two  
3. ...  
  
**Logs:**  
[Relevant log excerpts]  
  
**Expected Behavior:**  
[What should happen]  
  
**Actual Behavior:**  
[What actually happens]
```